MODULE 22 · EXAM STRATEGY - ALL BLOCKS

Spot the error

A three-step method for every 'what happens?' question, the signals of a syntax error, and twelve mixed drills.

BLOCKS all four FOCUS errors MODULE 22 of 28

admin@sthithapragna.com

Before tracing any code, ask three questions in order. **Does it compile?** Does it run to the end? Only then: what does it print?

Three passes over the code

01

1. Grammar

Colons, brackets, quotes, indentation, keywords, placement. Any fault: nothing prints.

›

02

2. Run

Types, indexes, keys, names, zero, argument counts. Find the first failing line.

›

03

3. Output

Everything printed before the failing line appears; nothing after it does.

›

04

4. Answer

Match the exception by name. Messages are clues, not answers.

THE KEY DIFFERENCE

A syntax error means no output at all. A runtime error means partial output, then the exception.

Syntax errors in statements

SIGNAL	EXAMPLE
missing colon	if count > 2
unclosed bracket or quote	print("hi"
bad indentation, or a block with only a comment	if x: # nothing else
keyword used as a name	class = 3
= in a condition	if x = 1:
else if instead of elif	else if x > 3:
break or continue outside a loop	if x > 2: break
return outside a function	return 5 at the top level

> ON THE EXAM
>
> Every row stops the whole file before line 1 runs.

Syntax errors in functions and exception handling

SIGNAL	EXAMPLE
def without brackets	def fun:
a default before a plain parameter	def f(a=1, b):
positional argument after a keyword argument	f(a=1, 2)
bare except: that is not last	except: then except ValueError:
global after the name is used	print(x) then global x
else with no except	try: … else: …
several exceptions without brackets	except A, B: before Python 3.14

ON THE EXAM

Again, nothing runs. Watch especially for argument order and bare except placement.

Runtime symptoms and the exception they raise

SYMPTOM	EXCEPTION
`str + int; calling a non-function; wrong argument count`	TypeError
`right type, unusable value: int("x"), [1].remove(9)`	ValueError
`position past the end of a list, string or tuple`	IndexError
`missing dictionary key`	KeyError
`name that does not exist`	NameError
`local read before it is assigned`	UnboundLocalError
`method the object does not have`	AttributeError
`divide or remainder by zero`	ZeroDivisionError

ON THE EXAM

Runtime errors happen only when their line is reached, so earlier output has already appeared.

How much gets printed before the error

SYNTAX ERROR ON LINE 3

Nothing at all

```
1  print("one")
2  print("two")
3  if True print("three")
```

Output: nothing. SyntaxError before line 1 runs.

RUNTIME ERROR ON LINE 3

Everything before it

```
1  print("one")
2  print("two")
3  print("three" + 3)
```

Output: one, two, then TypeError. Line 3 is where it stops.

THE TIEBREAKER

If an option says some lines print and then an error occurs, the error must be a runtime error.

Code that looks wrong, but runs

CODE	WHAT ACTUALLY HAPPENS
`code after a return`	legal; it simply never runs
`except Exception: before except ValueError:`	legal; the second branch is dead
`"a" == 1`	False, not an error
`x = x == x`	x becomes True
`[1, 2][1:9]`	[2]: slices never raise IndexError
`for i in range(3, 3): ... else: ...`	zero passes; the else still runs
`f(b=3, a=2)`	legal: keywords in any order
`a function defined below the one that calls it`	fine, if both defs run before the call

THE TRAP

Before choosing 'the code is erroneous', make sure the rule you think is broken really exists.

Error messages that point at the answer

- Since 3.10: NameError and AttributeError often add **Did you mean: ...?**
- Since 3.11: markers under the line show the exact part that failed.
- Python 3.14: keyword typos get **Did you mean 'while'?**, and string errors explain escaped quotes.
- Use these when you practise. On the exam, answer with the exception's name.

What to carry into the exam

- Pass 1: grammar. Any fault means **nothing prints**.
- Pass 2: run. Find the **first** line that raises, and name the exception.
- Pass 3: output. Everything before the failing line has printed.
- A handled exception is not an error: check for try blocks that catch it.
- Do not choose 'erroneous' unless you can name the rule that is broken.

What happens when this program runs?

```
1  print("Total")
2  count = 3
3  if count > 2
4      print("big")
```

A Total, then big

B Total, then a SyntaxError

C Nothing is printed; the program raises a SyntaxError

D Total only

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  print("Total")
2  count = 3
3  if count > 2
4      print("big")
```

SyntaxError: expected ':'. Found on the grammar pass, before line 1 runs.

A Total, then big

The if line has no colon.

B Total, then a SyntaxError

Syntax errors stop everything, including line 1.

C Nothing is printed; the program raises a SyntaxError

The missing colon is a grammar fault, so no line runs.

D Total only

Python never skips a broken line and carries on.

THE SIGNAL

Pass 1 finds it: a missing colon means nothing prints.

What happens when this program runs?

```
1  items = ["a", "b"]
2  print(len(items))
3  print("Count: " + len(items))
4  print("end")
```

A 2, then Count: 2, then end

B 2, then a TypeError

C Nothing is printed; the program raises a TypeError

D 2, then a ValueError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  items = ["a", "b"]
2  print(len(items))
3  print("Count: " + len(items))
4  print("end")
```

TypeError: can only concatenate str (not "int") to str.

A 2, then Count: 2, then end
+ never converts an int to a str for you.

B 2, then a TypeError
Line 2 runs; line 3 joins a str and an int, which is a type problem.

C Nothing is printed; the program raises a TypeError
A TypeError is a runtime error, so line 2 has already printed.

D 2, then a ValueError
The problem is the type of len's result, not its value.

THE SIGNAL

Pass 2 finds it on line 3. Pass 3: line 2 has already printed.

What happens when this program runs?

```
for i in range(3, 3):
    print("loop")
else:
    print("else")
```

A loop, then else

B Nothing is printed

C The code raises a SyntaxError

D else is printed

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  for i in range(3, 3):
2      print("loop")
3  else:
4      print("else")
```

range(3, 3) is empty: zero passes. No break, so the else runs.

A loop, then else
An empty range gives zero passes.

B Nothing is printed
The loop's else runs even after zero passes.

C The code raises a SyntaxError
else on a for loop is legal.

D **else is printed**
Zero passes, no break: only the else runs.

THE SIGNAL

Looks odd, runs fine. An empty range and a loop else are both legal.

What happens when this program runs?

```
1  def f(a, b=2):
2      return a * b
3
4  print(f(b=3, a=2), f(4))
```

A 6 8 is printed

B SyntaxError: positional argument follows keyword argument

C The code raises a TypeError

D 6 4 is printed

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  def f(a, b=2):
2      return a * b
3
4  print(f(b=3, a=2), f(4))
```

f(b=3, a=2) is 2 * 3. f(4) uses the default b = 2: 4 * 2.

A. 6 8 is printed
Keywords in any order are legal; the second call uses the default.

B. SyntaxError: positional argument follows keyword argument
Every argument in the first call is a keyword. No positional follows one.

C. The code raises a TypeError
Both calls supply a, and b is filled or defaulted.

D. 6 4 is printed
Forgets that f(4) still multiplies by the default 2.

THE SIGNAL

Keyword arguments may come in any order. Only a positional after a keyword is illegal.

What happens when this program runs?

```
1  stock = {"tea": 2}
2  print(stock.get("jam", 0))
3  print(stock["jam"])
```

A 0, then 0

B 0, then None

C 0, then a KeyError

D Nothing is printed; the program raises a KeyError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  stock = {"tea": 2}
2  print(stock.get("jam", 0))
3  print(stock["jam"])
```

get() never raises. Square brackets on a missing key do.

A 0, then 0
Square brackets have no default.

B 0, then None
Only get() returns None. Square brackets raise.

C 0, then a KeyError
Line 2 uses get's default; line 3 fails on the missing key.

D Nothing is printed; the program raises a KeyError
A KeyError is a runtime error, so line 2 has printed.

THE SIGNAL

get with a default is safe. Square brackets on a missing key are a KeyError.

What happens when this program runs?

```
1  total = 0
2  def add(n):
3      total += n
4
5  add(5)
6  print(total)
```

A 5 is printed

B 0 is printed

C The code raises an UnboundLocalError

D The code raises a SyntaxError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  total = 0
2  def add(n):
3      total += n
4
5  add(5)
6  print(total)
```

+= assigns total, so total is local in add, and it is read before it has a value.

A 5 is printed

Would need global total inside add.

B 0 is printed

The call to add fails before print is reached.

C The code raises an UnboundLocalError

total is local to add, and += reads it first.

D The code raises a SyntaxError

The grammar is fine. The problem only appears when add runs.

THE SIGNAL

Module 18's rule: assigning a name anywhere in a function makes it local there.

What happens when this program runs?

```
1  x = 3
2  print(x)
3  if x > 2:
4      break
```

A 3 is printed

B Nothing is printed; the program raises a SyntaxError

C 3, then a SyntaxError

D 3, then a RuntimeError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  x = 3
2  print(x)
3  if x > 2:
4      break
```

SyntaxError: 'break' outside loop.

A 3 is printed

break outside a loop is a grammar fault, so nothing runs.

B Nothing is printed; the program raises a SyntaxError

break must be inside a loop, and grammar is checked first.

C 3, then a SyntaxError

A syntax error stops line 2 as well.

D 3, then a RuntimeError

It is found before running, not while running.

THE SIGNAL

break outside a loop is grammar, not behaviour. The file never starts.

What happens when this program runs?

```
1  point = (1, 2)
2  x, y = point
3  print(x + y)
4  point[0] = 5
```

A 3, then a TypeError

B 3, then an AttributeError

C 3 is printed, and the program ends normally

D Nothing is printed; the program raises a SyntaxError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  point = (1, 2)
2  x, y = point
3  print(x + y)
4  point[0] = 5
```

TypeError: 'tuple' object does not support item assignment.

A. 3, then a TypeError
Unpacking and printing work; assigning to a tuple element does not.

B. 3, then an AttributeError
That is for a missing method, like append. This is item assignment.

C. 3 is printed, and the program ends normally
Tuples cannot be changed.

D. Nothing is printed; the program raises a SyntaxError
Assigning to an element is grammatical. It fails when it runs.

THE SIGNAL

Tuple element assignment: TypeError. A list-only method on a tuple: AttributeError.

What happens when this program runs?

```
1  x = 5
2  x = x == 5
3  print(x, type(x))
```

A `5 <class 'int'>`

B `False <class 'bool'>`

C The code raises a SyntaxError

D `True <class 'bool'>`

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  x = 5
2  x = x == 5
3  print(x, type(x))
```

The comparison x == 5 is worked out first: True. Then x is assigned True.

A `5 <class 'int'>`
x is reassigned on line 2, to the comparison's result.

B `False <class 'bool'>`
x is 5 when the comparison runs, so it is True.

C The code raises a SyntaxError
= then == on one line is legal: assign the result of a comparison.

D `True <class 'bool'>`
5 == 5 is True, and that bool is assigned to x.

THE SIGNAL

Looks odd, runs fine. The right side is evaluated completely before the assignment.

What happens when this program runs?

```
try:
    value = int("7") + "1"
except ValueError:
    print("value")
except:
    print("other")
```

A value is printed

B other is printed

C 8 is printed

D The program stops with an unhandled TypeError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  try:
2      value = int("7") + "1"
3  except ValueError:
4      print("value")
5  except:
6      print("other")
```

int("7") succeeds. 7 + "1" is a TypeError, caught by the bare except.

A value is printed

int("7") works. The failure is int + str, a TypeError.

B other is printed

The TypeError skips the ValueError branch and is caught by the bare except.

C 8 is printed

An int and a str are never added.

D The program stops with an unhandled TypeError

The bare except catches everything that reaches it.

THE SIGNAL

Name the exception, then check the handlers. A caught exception is not an error.

What happens when this program runs?

```
1  names = ["Ann", "Bob"]
2  for i in range(len(names) + 1):
3      print(names[i])
```

A Ann, then Bob

B Nothing is printed; the program raises an IndexError

C Ann, then Bob, then a KeyError

D Ann, then Bob, then an IndexError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  names = ["Ann", "Bob"]
2  for i in range(len(names) + 1):
3      print(names[i])
```

range(3) gives 0, 1, 2. names[2] does not exist.

A Ann, then Bob
The loop makes a third pass, with i = 2.

B Nothing is printed; the program raises an IndexError
The first two passes succeed and print before the error.

C Ann, then Bob, then a KeyError
Lists raise IndexError. KeyError is for dictionaries.

D Ann, then Bob, then an IndexError
Passes 0 and 1 print; pass 2 is past the end.

THE SIGNAL

Off by one in a range is a runtime IndexError, so earlier passes have already printed.

> Each line is saved and run as a whole program on its own.

Which two programs raise a SyntaxError, so that nothing runs? (Choose two.)

A `def f(a=1, b): pass`

B `print(undefined_name)`

C `x = [1, 2]; print(x[5])`

D `while True print(1)`

◆ PAUSE AND DECIDE

Which two programs raise a SyntaxError, so that nothing runs? (Choose two.)

A `def f(a=1, b): pass`

A plain parameter after a default. SyntaxError.

B `print(undefined_name)`

Grammatical. NameError when it runs.

C `x = [1, 2]; print(x[5])`

Grammatical. IndexError when it runs.

D `while True print(1)`

Missing colon. SyntaxError.

THE SIGNAL

Grammar faults: SyntaxError. Faults that depend on values or names: runtime exceptions.